电路模型探究装置 • 固件实现

Keil 手动建工程指南

不用 CubeMX、不用 HAL，只用寄存器

配套代码：固件_裸机/（Core 2 个、BSP 2 个、App 8 个，共 12 个文件）

目标芯片：STM32F407VET6（168 MHz，512 KB Flash，LQFP100）

工具链：Keil MDK-ARM 5.38 ＋ STM32F4xx_DFP ＋ STM32CubeProgrammer（DFU 烧录）

2026 年 10 月

1 这套代码是什么

不用 CubeMX，也不用 ST 的 HAL 库。整个固件只有 12 个文件，直接读写 STM32F407 的寄存器，编译时除了 Keil 自带的 CMSIS 头文件和启动文件，不需要任何第三方库。

表 1 与 CubeMX 版的区别

对比项	CubeMX + HAL 版	本裸机版
依赖	STM32CubeF4 固件包（几十 MB）	只要 Keil 的 STM32F4xx_DFP 器件包
工程生成	CubeMX 图形化配置后生成	工程已建好，双击 .uvprojx 直接打开
初始化代码	MX_xxx_Init() 一堆函数	bsp.c 里 5 个函数，逐行可读
代码量	编译产物 40 KB 以上	约 12 KB
可解释性	HAL 内部做了什么看不到	每个寄存器写的是什么一目了然，答辩好讲

代价是：时钟树、DMA 请求映射、NVIC 这些必须自己配对。下面每一处配置都给了寄存器含义，照着写不会错；写错了第 4 章有对应的排查表。

2 准备清单

2.1 先装器件包：没有它，Device 列表里根本没有 STM32F4

µVision 安装包本身只带 ARM Compiler 与 CMSIS 内核头文件，不含任何厂商的器件支持。STM32F4 的芯片型号、寄存器定义、启动文件、Flash 下载算法，全部由 Keil.STM32F4xx_DFP 这一个包提供。所以新建工程时如果 Device 列表里搜不到 STM32F407，不是板子的问题，是这个包没装。

三种装法，按顺序试：

① Pack Installer（有网最省事）：µVision 工具栏点 Pack Installer 图标 → 左侧 Devices 展开 STMicroelectronics → STM32F4 Series，右侧 Packs 页找到 Keil::STM32F4xx_DFP → 点 Install。国内网络常卡在 Updating pack index 不动，等五分钟没反应就换 ②。

② 离线 .pack（推荐，最稳）：已随本指南放在 D:\Agent\project\电路模型探究装置\工具\Keil.STM32F4xx_DFP.2.17.0.pack（276 MB），双击即可，µVision 会自动弹出安装进度；装完关掉再重开 µVision。

③ 手动解压：把 .pack 当 zip 解开，整个文件夹放到 Keil_v5\ARM\PACK\Keil\STM32F4xx_DFP\2.17.0\，确认该目录下能看到 Keil.STM32F4xx_DFP.pdsc，重开 µVision 就能识别。

> **别装 3.x** keil.arm.com 上最新版是 3.1.1，只有 2 MB。3.0.0 起为配合 CMSIS-Toolbox 与 CubeMX 重构，把 CMSIS Drivers、器件头文件、启动文件全删了，只留 Flash 算法与 SVD。装完 3.x 仍然找不到 stm32f4xx.h 和 startup_stm32f407xx.s，本工程必须用 2.17.0。

验证：Project → New uVision Project → Device 页搜索 STM32F407，能出现 STM32F407VE（Flash 512 KB）就说明装好了，继续往下走。

2.2 软硬件清单

表 2 软硬件清单

类别	项目	说明
软件	Keil MDK-ARM 5.38+	编译器用 ARM Compiler 6
软件	STM32F4xx_DFP 2.17.0	见 2.1 节；提供器件头文件、startup、512 KB Flash 算法
软件	STM32CubeProgrammer	只用它的 DFU 下载功能，不用它的代码生成
硬件	开发板	STM32F407VET6（你这块丝印 STM32_F4VE）
硬件	杜邦线 2 根	短接 BT0 与 3V3，进 DFU 用
硬件	Type-C 数据线	板子唯一的 USB，直连 PA11/PA12
硬件	USB-TTL 模块（可选）	接 PA2/PA3 看 printf；没有就先靠示波器

3 打开工程（或从零重建）

工程已经建好了 直接双击 D:\Agent\project\电路模型探究装置\CircuitModel.uvprojx 就能打开，按 F7 编译出 Objects\CircuitModel.hex。3.1～3.5 是从零重建的完整步骤——工程文件丢了、或者要在另一台电脑上重做时照着走一遍，顺带说明每个选项为什么这么设。

3.1 工程目录长什么样

工程根目录 D:\Agent\project\电路模型探究装置 下的结构如下。注意本工程不用 RTE：器件头文件、内核头文件、启动文件都已经拷进 Drivers\ 目录，工程自带，换台电脑也能直接编。

表 3 分组与文件对应关系

分组	所在目录	加入的文件	作用
User	Core	main.c、system_stm32f4xx.c	主程序与系统文件
BSP	BSP	bsp.c	时钟、串口、定时器、DAC/ADC + DMA
App	App	app_dds.c、app_meas.c、app_known.c、app_ident.c	四个功能模块
Drivers	Drivers\CMSIS\Device\Source	startup_stm32f407xx.s	中断向量表与复位入口

头文件目录不加入工程，只挂在 Include Paths 上：Drivers\CMSIS\Device\Include 放 stm32f4xx.h、stm32f407xx.h、system_stm32f4xx.h；Drivers\CMSIS\Include 放 core_cm4.h 及其依赖的 8 个 CMSIS 头文件。这两批文件分别从 Keil.STM32F4xx_DFP 2.17.0 和 ARM.CMSIS 5.9.0 里取，已经随工程打包好。

3.2 建空工程、选器件

Keil 菜单 Project → New uVision Project…，目录就选 D:\Agent\project\电路模型探究装置，文件名填 CircuitModel，保存。

弹出 Select Device for Target 对话框：展开 STMicroelectronics → STM32F4 Series → STM32F407，选 STM32F407VE（注意是 VE 不是 VG，Flash 512 KB），点 OK。

紧接着会弹出 Manage Run-Time Environment —— 本工程一个都不勾，直接点 Cancel 关掉。因为启动文件和系统文件已经放进 Drivers\ 和 Core\ 了，再勾一次会重复定义。

3.3 建分组、加源码

Project 窗口右键 Target 1 → Manage Project Items…，按表 3 新建四个组，然后逐个 Add Files。

加完检查一下：Groups 树里应该有 7 个 .c 文件 + 1 个 .s 文件，少一个都链接不过。

3.4 Options for Target 六页设置

点魔术棒图标或按 Alt+F7，逐页按表 4 设置。这六项里任何一项漏了，都会在第 4 章对应到某个具体报错。

表 4　Options for Target 设置

页面	设置项	值	漏了会怎样
Target	ARM Compiler	Use default compiler version 6	AC5 也能编，但语法检查更松
Target	Use MicroLIB	勾选	printf 卡死或链接报 _sys_write
Target	Floating Point Hardware	Single Precision	进 main 就 HardFault
Output	Create HEX File	勾选	Objects 里没有 hex，无法 DFU 烧录
C/C++	Define	STM32F407xx	报 #error Please select first the target device
C/C++	Include Paths	.\Core;.\BSP;.\App;.\Drivers\CMSIS\Device\Include;.\Drivers\CMSIS\Include	报 stm32f4xx.h / core_cm4.h 找不到
C/C++	Optimization	Level 1 (-O1)	-O0 也能跑，辨识慢一点
C/C++	C99 Mode	勾选	for 内定义变量会报错
Linker	Use Memory Layout from Target Dialog	勾选	报 Scatter Error: no default 'Read/Write' range selected

Linker 那一项勾上之后，链接脚本由 Target 页的 Read/Write Memory Areas 自动生成：IROM1 = 0x08000000 / 0x80000（512 KB Flash），IRAM1 = 0x20000000 / 0x20000（128 KB SRAM）。工程里另附了一份 CircuitModel.sct，万一那张表空了，可以在 Linker 页取消勾选、把 Scatter File 指向它。

Debug 页在没有 ST-Link 时随便选（比如 Use Simulator），它只影响“点调试按钮”的行为，不影响编译和 hex 产出。

3.5 第一次编译

按 F7（或 Project → Build Target）。底部 Build Output 应显示 0 Error(s)。编译完在 Objects\CircuitModel.hex 就能看到 —— 这就是要烧进芯片的文件。

如果报错，先看第 4 章的表 5，那几条覆盖了新手 90% 的问题。

4　上手必踩的坑

表 5　编译与启动阶段的典型问题

现象	原因	处理
L6218E: Undefined symbol SystemInit	只加了启动文件，没加系统文件	把 Core/system_stm32f4xx.c 加进 User 组
L6200E: Symbol SystemInit multiply defined	既加了 Core 里的系统文件，又在 RTE 里勾了 Device:Startup	RTE 全部取消勾选
#error Please select first the target STM32F4xx device	C/C++ → Define 里没写器件宏	填 STM32F407xx
下载后一上电就 HardFault / 卡死	FPU 没开：Target 页 Floating Point 仍是 Not Used	改成 Single Precision 重新编译
printf 没有任何输出	MicroLIB 没勾，或 fputc 被删	Target 页勾 Use MicroLIB；确认 main.c 里的 fputc 还在
编译报 stm32f4xx.h / core_cm4.h: No such file	Include Paths 漏了 Drivers 下的两个目录	补上 .\Drivers\CMSIS\Device\Include 与 .\Drivers\CMSIS\Include
注释全是乱码	Keil 按 GBK 读了 UTF-8 文件	Edit → Configuration → Editor → Encoding 选 UTF-8，重开文件

5　下载：BT0 短接进 DFU

这块板没有 ST-Link 接口可用，但 STM32F407 片内有 ROM bootloader，支持 USB DFU。板子的 Type-C 直连 PA11/PA12，正好就是 DFU 用的那两个脚。

表 6　烧录步骤

步骤	操作	判断成功的标志
1	用杜邦线把上排的 BT0 针与旁边的 3V3 针短接	两根针之间导通
2	按一下左上角的复位键（没有就拔插一次 USB）	设备管理器出现 STM32 BOOTLOADER（0483:DF11）
3	打开 CubeProgrammer，连接方式选 USB，Port 选 USB1，点 Connect	右侧显示 STM32F407xx、Flash 512 KB
4	Download 页选 Objects\CircuitModel.hex，点 Download	日志出现 Download verified successfully
5	拔掉杜邦线，按复位	程序开始运行

第 5 步不能省　BT0 接着 3V3 时，芯片每次复位都回到 bootloader，程序永远起不来。拔线再复位，才算真正运行用户程序。

6　上电验证（五级）

按顺序逐级验证，任何一级不过就停在那里查，不要跳步。

表 7　五级验证

步骤	操作	预期现象	不通过先查
1 串口	USB-TTL 接 PA2/PA3，115200	打印 ==== Circuit Model Explorer ====	TX/RX 接反、波特率、BRR 计算
2 DAC	输入 F1000 回车，再 A1.0 回车，示波器看 PA4	1 kHz、1 Vpp、偏置 1.65 V 正弦	TIM6 是否启动、DAC DMAEN 是否置位

步骤	操作	预期现象	不通过先查
3 ADC	PA4 用杜邦线短接 PA0，输入 M	读到约 1.0 Vpp	ADC 触发源是否 TIM3_TRGO、DMA 是否使能
4 开环	接已知模型电路，输入 C1.5	打印目标/实测/误差，误差 ≤3%	电路增益、kcal 校准
5 辨识	接未知 RLC 电路，输入 L	约 1～3 s 后打印类型与 K、f0、Q	激励是否 ≥1.5 Vpp

7 常见错误（运行阶段）

表 8 运行阶段问题

现象	原因	处理
DAC 输出恒定 1.65 V	TIM6 没启动，DMA 收不到请求	检查 tim6_start() 是否在 dac_dma_start() 之后调用
DAC 输出恒定不变、没有任何波形	DAC1_CH1 的请求线硬件固定为 DMA1_Stream5_Ch7，若配成 DMA2 则永远等不到请求	改用 DMA1_Stream5，AHB1ENR 开 DMA1EN；Stream5 标志位在 HISR 的 bit10(HT)/bit11(TC)
输出频率正好差一倍	ARR 与定时器时钟不匹配	TIM6：ARR = 84 MHz/1 MHz − 1 = 83
波形有台阶、毛刺	重构低通没接或截止太高	接 150 kHz 二阶 Sallen-Key 重构滤波器
串口乱码	晶振不是 8 MHz 或 BRR 算错	BRR = PCLK1/baud ≈ 365（42 MHz/115200）
测量值随频率大幅漂移	ADC 采样时间太短、前端驱动不足	SMPR2 从 3 周期改到 15 周期
辨识恒为 Low-pass	激励太小、噪声主导	A 设到 1.5~2 Vpp 再 L
L 命令卡住不动	扫频等待时间累加	26 点 × max(20/f,5 ms) ≈ 0.34 s，加四模型网格拟合共 1～3 s，属正常

8 代码结构与运行流程

表 9 文件职责

文件	职责	关键点
Core/main.c	初始化顺序、串口命令行、printf 重定向	fputc 直接写 USART2->DR
Core/system_stm32f4xx.c	使能 FPU、SystemCoreClock	不做时钟配置，只补 startup 需要的符号
BSP/bsp.c	时钟 168 MHz、SysTick、USART2、TIM6/TIM3、DAC/ADC + DMA	所有寄存器配置集中在这里
App/app_dds.c	相位累加、查表插值、DMA 双缓冲重填	在 DMA1_Stream5 中断里填另一半
App/app_meas.c	ADC 采集 + Hann 窗单频 DFT	采满 4096 点由 DMA 中断置标志
App/app_known.c	H(s) 幅相、开环反算 Vin	纯数学，不碰硬件
App/app_ident.c	26 点扫频 + 四模型拟合 + 分类	网格粗搜 + 局部细化

两条数据流是并行的，互不阻塞：

① 输出链：TIM6（1 MHz 更新）→ DAC1_CH1 → DMA1_Stream5 循环搬运 2048 点 → PA4。每搬完一半进一次中断，把刚空出来的 1024 点按当前频率重算。

② 输入链：TIM3（500 kHz）→ ADC1 触发 → DMA2_Stream0 搬 4096 点 → 采满进中断 → 主循环算 DFT。

主循环本身只做一件事：收串口命令。所以 DAC 输出和 ADC 采集都不会被命令处理打断。

9　与设计报告的一致性

表 10　关键参数对照

参数	代码中的值	报告出处
系统时钟	168 MHz（PLL M=8 N=336 P=2）	4.1 节 主控与资源分配
DAC 更新率	1 MSPS（TIM6 ARR=83）	4.2 节 正弦信号发生通道
正弦表	1024 点 Q15 + 线性插值	4.2 节
ADC 采样率 / 点数	500 kSPS（TIM3 ARR=167）/ 4096 点	4.3 节 幅相测量通道
窗函数	Hann，幅度按 2/Σw 修正	5.3 节 单频 DFT
扫频点	26 点，200 Hz~100 kHz 对数分布	5.5 节 未知模型辨识
稳定时间	max(20/f, 5 ms)	5.5 节
辨识模型	LP/HP/BP/BR 四模型取最小残差	5.5 节

答辩小抄　被问到“为什么 DAC 用 TIM6 而不是软件触发”，答：软件触发频率抖动大，1 MSPS 下抖动几十纳秒就会让输出频率误差到 0.1% 量级；定时器硬件触发把抖动压到时钟周期级。